EXPERIMENT 12

Implementing Security Features and Governance / Administration Practices in Zoho Creator

Application: Patient Appointment Scheduling System | Zoho Creator

Saranya V. | Register No: 2116241501188 | AIML | Rajalakshmi Engineering College

Abstract

This experiment reviews security and governance practices for the Patient Appointment Scheduling System in Zoho Creator. The actual application screenshots show user permission sets, the Add Permission Set dialog, the Appointment Booking Form and the environment/deployment menu. The report explains role-based access, mandatory-field validation, environment separation and additional administrative practices for protecting patient information.

1. Aim

To implement and review security features, governance and administration practices in the Patient Appointment Scheduling System built using Zoho Creator.

Execution Note

The supplied screenshots confirm that the permission controls, appointment form and deployment menu are available. They do not, by themselves, prove that every permission rule, mandatory-field validation test, private link, two-factor authentication setting or audit-log review has been completed. Where a step has not been verified by the screenshots, it is presented as a procedure to perform rather than as a completed action.

2. Concepts

1. Access security and permission sets

The User Permissions page defines permission sets for application users. The Add Permission Set dialog contains module-level controls such as Access, View, Edit, Delete and More. Roles and Data Sharing settings can be used to manage record visibility where available.

2. Data validation

Mandatory fields help prevent incomplete appointment records from being submitted. In this project, important fields may include Patient Name, Email Address, Contact Number, Symptoms Description, Preferred Appointment Date and Preferred Appointment Time. Configure required fields based on the form design and test the live form by leaving them empty.

3. Governance and environment separation

Development is used to build changes, Stage is used for pre-release testing, and Production is the live environment. Test updates before promoting them to Production. Use consistent names for forms, fields and workflows.

4. Patient-data protection

Patient contact details and symptom descriptions should only be visible to users who need them. Use least-privilege permissions, avoid unnecessary public access to forms, enable two-factor authentication for the Zoho account, and review activity logs where available.

3. Procedure

1. Open the Patient Appointment Scheduling System in Zoho Creator and select Settings → User Permissions.

2. Review the Permissions, Roles and Data Sharing tabs. Identify the permission sets already present and the modules they apply to.

3. Open Add Permission and review the Access, View, Edit, Delete and More controls for Appointment Booking Form, Doctor Master, Family Member and Dashboard.

4. Create or adjust role-specific permission sets according to the user's duties. For example, a receptionist may create appointments, a doctor may view relevant appointment details, and an administrator may manage the application. Grant only the permissions needed.

5. Open Design → Appointment Booking Form. Review Field Properties and enable Mandatory for fields that must be supplied before an appointment can be submitted.

6. Open the live form and attempt a test submission with a required field empty. Confirm that the form blocks the submission and displays a validation message. Record this step as completed only after testing it in the live app.

7. Use the application menu to review Development Live, Stage Live and Production Live. Test changes in Development and Stage before promoting a release to Production, where deployment is enabled.

8. Review form sharing settings. Prefer access restricted to intended users instead of an open public link when the form contains patient information.

9. Enable two-factor authentication for the Zoho account and review activity/audit logs if these options are available in the account.

10. Test with a non-administrator account to confirm that users cannot view, edit or delete records beyond their assigned permissions.

4. Security and Governance Checklist

• Permission sets are assigned according to the user's role.

• Patient and appointment data are visible only to authorized users.

• Required fields block incomplete submissions when tested.

• Development and Stage changes are tested before Production deployment.

• Form sharing is limited appropriately for patient data.

• Two-factor authentication is enabled and activity logs are reviewed where available.

5. Implementation Screenshots

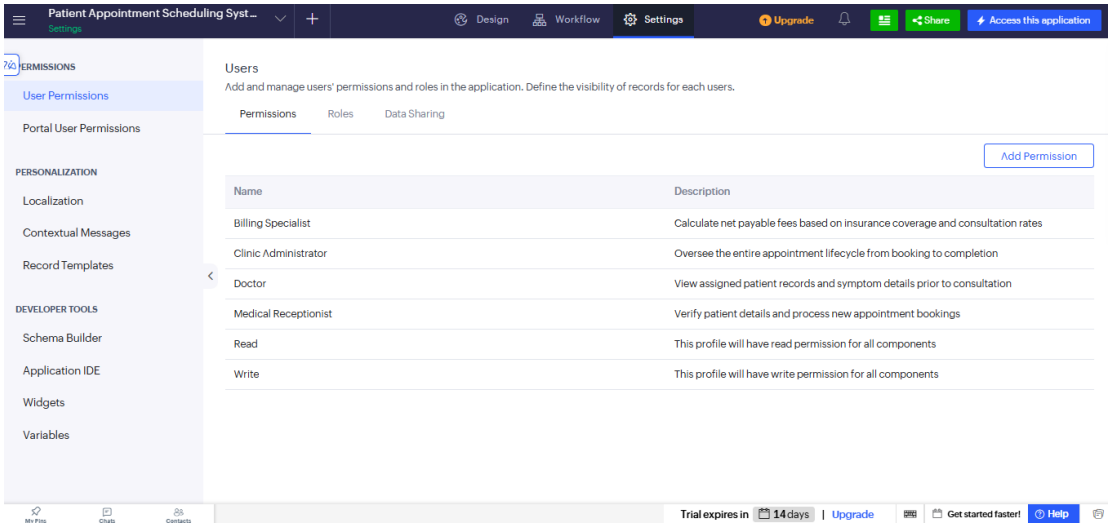


Figure 1: User Permissions page showing the available permission sets for the Patient Appointment Scheduling System.

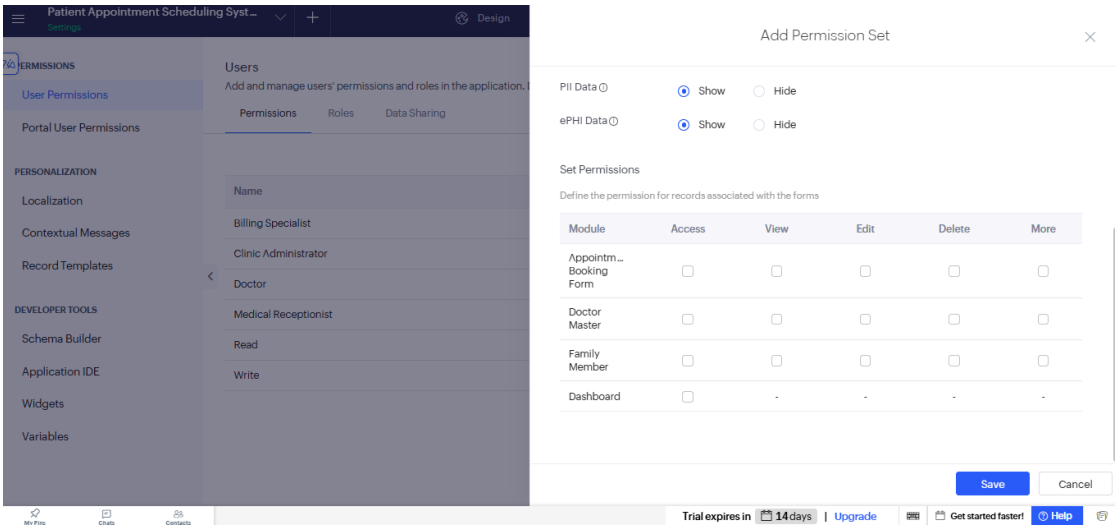


Figure 2: Add Permission Set dialog showing module-level Access, View, Edit, Delete and More controls.

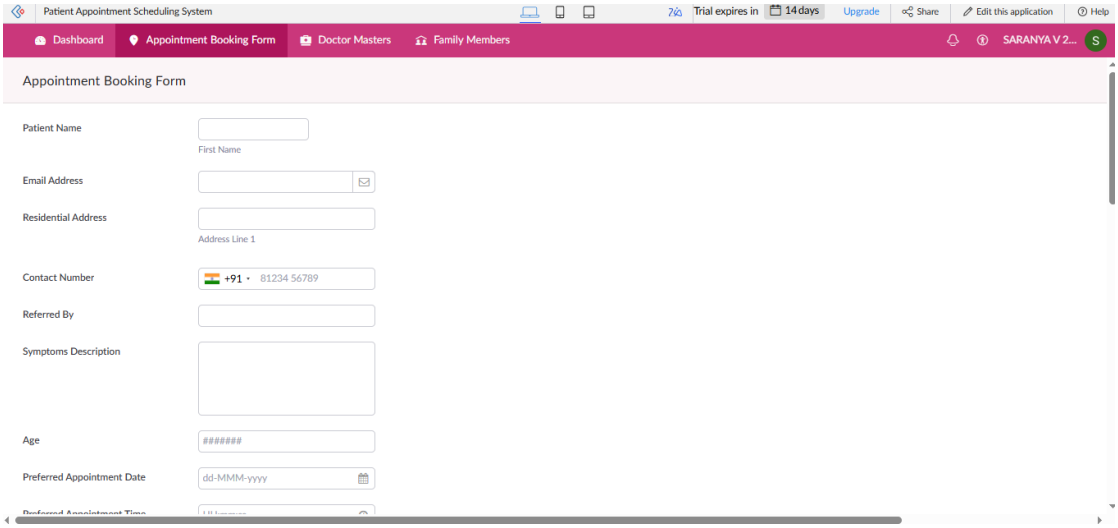


Figure 3: Appointment Booking Form containing patient details and appointment fields.

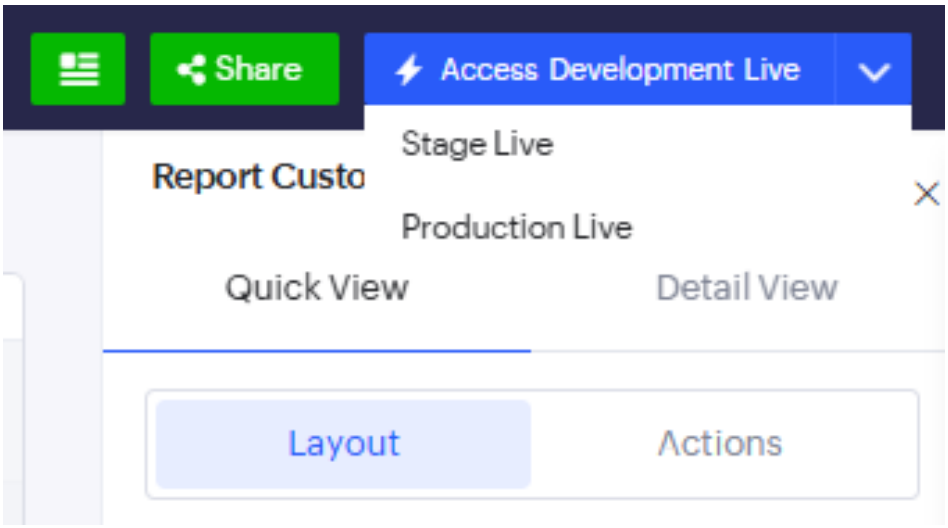


Figure 4: Application environment/deployment menu showing Development Live, Stage Live and Production Live options.

6. Expected Output

Only users with suitable permissions should be able to view or modify appointment records. The live form should reject incomplete submissions when required fields are left empty, and development changes should be tested before production use. Additional account-security and logging practices strengthen governance.

7. Result

Security and governance practices were adapted to the Patient Appointment Scheduling System using the available screenshots of permission settings, the appointment form and environment/deployment options. The report includes a procedure for validating mandatory fields and reviewing additional security practices. Any action not verified in the supplied screenshots should be tested in the live account before being reported as completed.

8. Learning Outcome

Learnt how permission sets and module-level access controls protect application data, how mandatory validation can prevent incomplete appointment submissions, and how environment separation and account-security practices support governance in a low-code application.